Week 9 - Midterm-test - Total 50 marks

Chap 1 - 20 mark

- All theory

Q1. **Figure 1** shows the interaction among computer system components.

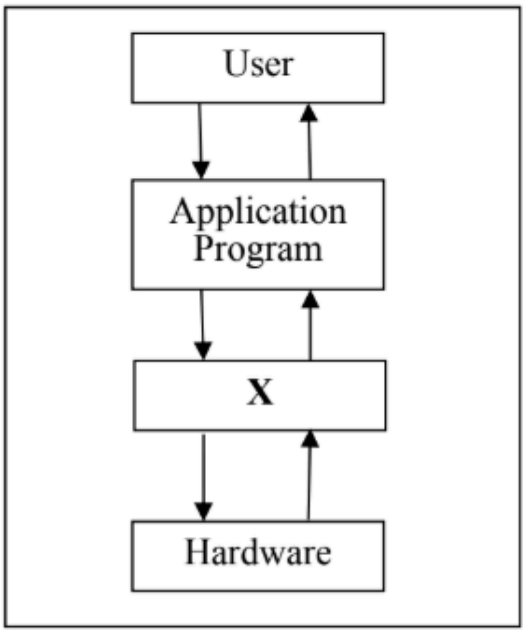


Figure 1: Interaction among computer system components

Identify and define component X.

X = Operating System

- A software that sits between the hardware and application program
- The most important software in a computer system

TWO (2) examples of OS in the market.

- Windows
- Linux

FOUR (4) functions which are performed by the OS.

- Make the computer system easy to use.
- Use the computer hardware in an efficient manner.
- Controls the execution of user programs and the operations of I/O devices.
- Allocate resources to specific programs

Characteristics of an OS

- Concurrency, Facilitates multiprogramming
- Sharing, Allows resources and information to be shared

Types of Operating Systems

- Batch systems: Process batches of similar jobs serially
- Embedded systems: Computers placed inside other products to add specific capabilities
- Interactive/ time-sharing systems: Support interactive users with quick response times
- Hybrid systems: A combination of batch and interactive systems
- Real Time systems: N/A

Four Components of a Computer System

- Hardware
- Operating System (OS)
- Application Programs
- Users

The key elements of OS. ???

- Memory manager: Checks validity of each request for memory space
- Processor manager: Allocates CPU time.
- Device manager: Monitors every device and control unit.
- File manager: Keeps track of each file in the system

Dual-mode operation two mode
User mode – Execution done on behalf of a user.
kernel mode – Execution done on behalf of the OS.

System Calls

- System calls provide a programming interface to the services provided by the OS

Two Categories of System Calls with Examples
- Process Control: Examples include fork(), exit(), and wait()
- File Manipulation: Examples include open(), read(), and write()

Traps VS Interrupts

Traps	Interrupts
It is a software-generated interrupt produced by a running program..	It is a hardware-generated signal sent to the CPU by a device
Triggered by internal program activities	Triggered by external hardware events

Chap 2 - 12 mark

- Calculations and (maybe will have little theory)

Theory part (Slides 1 - 26)
Differentiate between the following.
(a) I/O bound and CPU bound

I/O bound jobs	CPU-bound jobs
These jobs have many brief CPU cycles and long I/O cycles	These jobs have long CPU cycles and shorter I/O cycles

(b) Preemptive scheduling and non-preemptive scheduling

Preemptive scheduling	non-preemptive scheduling
This policy interrupts the processing of a job and transfers the CPU to another job.	This policy functions without external interrupts

Calculations - 6个会出其中3 type , 每个4分
过程表**format**

Process					
Arrival Time					

Preemptive 插队, non-preemptive是不插队的

没有 Priority 的版本

Q1. Given the following metrics (Table 1) that show the arrival time, CPU burst time (in millisecond) and priority of a list of jobs.

Table 1: List of processes going to execute

Process	Arrival Time	CPU Burst Time (ms)
A	0	10
B	3	5
C	5	3
D	7	8

Draw a Gantt chart to illustrate the execution of the above jobs, and calculate the average turnaround time and average waiting time based on each of the following algorithms:

a) First Come First Served (FCFS) - non preemptive

只看arrival time, 谁最靠近/最短时间 就继续谁

要画顺序表, 然后跟着表做table出来

	A	B	C	D
0	10	15	18	26

Table format

- Turnaround Time (TA) = Finish Time - Arrival Time
- Wait Time (WT) = Finish Time - CPU Cycle - Arrival Time

Process	Arrival Time	CPU Cycle	Finish Time	Turnaround Time	Wait Time
A	0	10	10	10	0
B	3	5	15	12	
C	5	3	18	13	
D	7	8	26	19	

b) Shortest Job Next (SJN) - non preemptive

看CPU cycle, 会先走完第一个,然后根据走了的arrival time看谁的CPU cycle最短就走谁的, 会走完整process才继续下一个

A	C	B	D
10	13	18	26

c) Shortest Remaining Time First (SRT) - preemptive

跟SJN是一样的format, 但是它不会走完整个process的, 会一直插队, 要自己记录每一次插队扣的cpu cycle

我在对应的step放它的happening, 考试不用写的

Step 1

- A开始嘛, 走10步, 但是到第3步的时候, 有一个B的是第3步来的

Step 2

- B的cycle跟短, 所以代替A（A/10 - 3 = A/7）, B是走5步, 但是3+1+1步的时候到第五步, C的第5步也到了。进度 : A = 7

Step 3

- 进度 : A = 7 B = 3 C = 3. B跟C都是走3步。所以继续走完3步。5+1+1 = 7步, 到7步的时候会遇到D, 但是它是8 所以不用理。进度: A = 7 B = 1 C = 3 D = 8

Step 4

进度: A = 7 B = 0 C = 3 D = 8

到这里的时候, 全部的arrival time已经走完了, 剩下的直接加下去, 没有队可以插队了

Step 5

走c三步, 8+1+1+1 = 11步 。进度: A = 7 B = 0 C = 0 D = 8

Step 6

直接走A 7步，11+7 = 18步. 进度: A = 0 B = 0 C = 0 D = 8

Step 7

直接走D 8步, 18+8 = 26步. 进度: A = 0 B = 0 C = 0 D = 0

step	1	2	3	4	5	6	7
	A	B	B	B	C	A	D
0	3	5	7	8	11	18	26

d) Round Robin - preemptive // (Assume that time quantum = ?ms)

队伍怎么来的**?** : 比如看这个表的Arrival time, if quantem=3ms, 所以一开始走3步, 只有A, B才在队伍里面(因为他们的arrival time是3以内) , 然后变成B, A, 然后再走B的3步, 第6步的时候C也是出来了(他的arrival time是5), 所以队伍变去A, C, B

Format: 根据它set的quantum时间=？, 只能走？步。每个process走完？步就排队去队尾

题目, Assume that time quantum = 3ms. so只能每次走3步

队伍状态	到第三步了, 所以A , B 都有在队伍里, 然后A排队尾. 队伍: B, A	到第6步了 , 所以A, B, C 都有在队伍里 , 然后B 排队尾. 队伍: A, C, B	到第9步了, 所以A , B, C, D都有在队伍里, 然后A排队尾. 队伍: C, B, D, A	到第12步了, 全部都有在队伍里, 然后C消失(因为没有了). 队伍: B, D, A	到第14步了, 全部都有在队伍里, 然后B消失(因为没有了). 队伍: D, A	继续上一次的队伍是:D, A。所以直接走D, 然后一样排去队尾。 队伍: A, D	继续上一次的队伍是A, D。所以直接走A, 然后一样排去队尾。 队伍: D, A	继续上一次的队伍是D, A。所以直接走D, 然后一样排去队尾。 队伍: A , D	继续上一次的队伍是A , D。所以直接走A, 然后一样排去队尾。 队伍: D	最后走完剩下的D
CPU cycle剩余	走A: 10 - 3 = 7	走B: 5 - 3 = 2	走A: 7 - 3 = 4	走C:3 - 3 =0	走B:2 - 2 = 0	走D:8 - 3 =5	走A: 4 - 3 =1	走D:5 - 3 =2	走A:1 -1 = 0	2 - 2 =0
	A	B	A	C	B	D	A	D	A	D
0	3	6	9	12	14	17	20	23	24	26

(a) Consider the following set of processes, with the length of the CPU burst given in milliseconds (ms). Ignore no context switch time.

Process	Arrival Time	Execution Time	Priority
P1	0	3	1
P2	2	6	2
P3	3	2	3
P4	6	4	3
P5	9	3	4

Execution time = CPU cycle = Burst time

Priority 是看题目讲Assume a larger/smaller number implies a higher priority
但是一开始都是0开始的

e) Priority preemptive

题目: Assume a larger number implies a higher priority

status	P1: 3-2=1	P2: 6-1=5	P3: 2-2=0	P2: 5-1=4	P4: 4-3=1	P5: 3-3=0	P4: 1-1=0	P2: 4-4=0	P1: 1-1=0
	P1	P2	P3	P2	P4	P5	P4	P2	P1
0	2	3	5	6	9	12	13	17	18

f) Priority non-preemptive

题目: Assume a larger number implies a higher priority

	P1	P3	P2	P5	P4
0	3	5	11	14	18

Chap 3 - Deadlock - 18 mark

- A few theory and two calculations

Theory part (3 mark)

Three Conditions for Deadlock

- Mutual Exclusion资源独占: non-sharable mode
- No Preemption不可抢占: Only holding process can release resource
- Circular wait 循环等待: Process A waits for Process B waits for Process C waits for Process A.

Deadlock VS Starvation

	Deadlock	**Starvation**
Define	A group of jobs are stuck in a "deadly embrace"	A single job is prevented from executing

Calculations part (15 mark)
Deadlock draw 6 mark + explain 2 mark（total 8 m）

Q2. (a) Consider a system with four processes P1, P2, P3 and P4 and three resources types A, B and C. Resource type A has 2 instances, resource type B has 3 instances and resource type C has 1 instance.

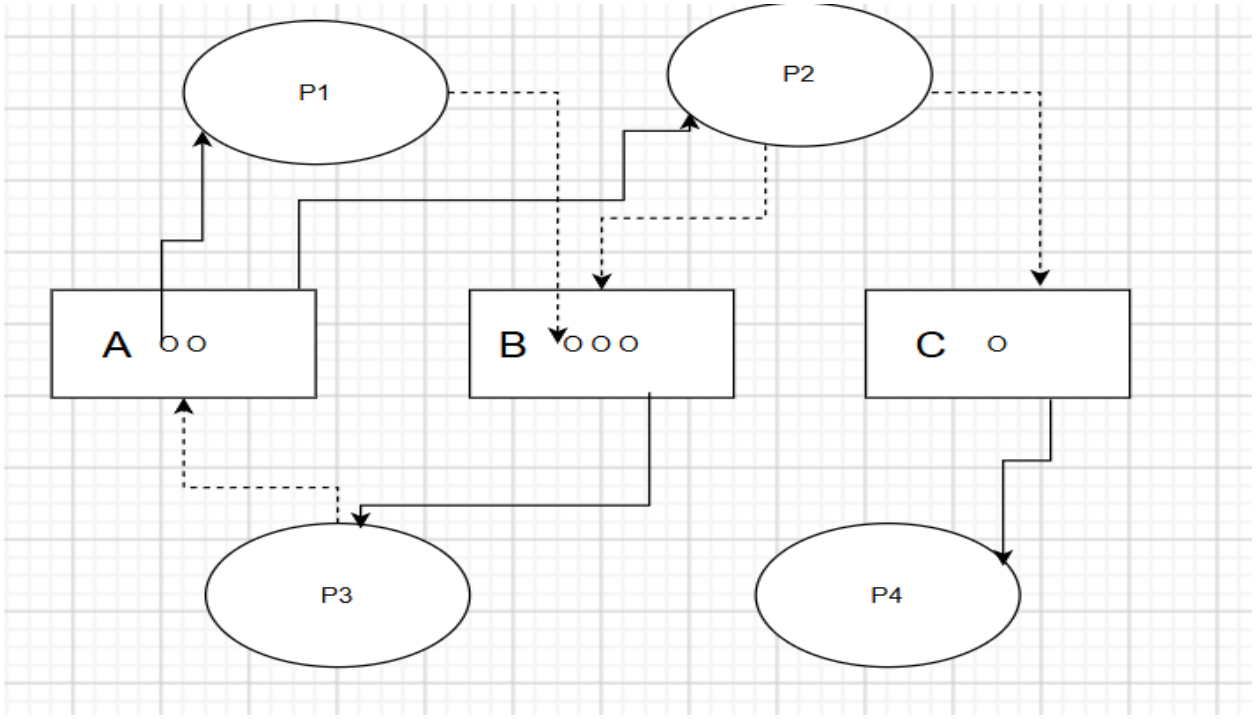
- P1 holds an instance of A and requests an instance of B.
- P2 holds an instance of A and requests an instance of B and C.
- P3 holds an instance of B and requests an instance of A.
- P4 holds an instance of C.

Draw a Directed Resources Graph for the above situation.

(b) Are these four processes mentioned in Q2. (a) in a deadlocked state? Justify your answer.

a) Draw deadlock graph

Format: P就是process, 画圆圈将的, instance of ?是画长方形，里面的instances就是画坤蛋(题目一定会给有多少个)；
P hold着那个instance(所以logic来说instance指过去P/属于P的) //
P request着那个instance(所以logic来说P指过去instance/需要instance)



b) Is there a deadlock? Explain

No deadlock.
P1 - P3 - P4 - P2
P4 - P1 - P2 - P3

Banker's algorithm check process能不能继续走下去 / q3（total 7 m）

Q3. Assuming that there are 5 processes (P1, P2, P3, P4 and P5) which are processed by a CPU. While processing, the processes need 3 resources (A, B, C). The **Table 1** below has shown the resources allocated to each current process, maximum resources the processes require to complete their tasks and the available resources respectively.

Use ***banker's algorithm*** to determine whether the system is in a safe state or unsafe state when the process sequence is P1→ P5. If the system is in an unsafe state, propose a process sequence where the system is in a safe state. Justify your answers.

	A	B	C
P1	0	1	2
P2	2	0	0
P3	3	2	1
P4	1	2	3
P5	0	4	2
Allocated resources			

	A	B	C
P1	0	3	4
P2	3	3	4
P3	5	6	2
P4	2	3	4
P5	1	6	6
Max resources required			

A	B	C
2	2	2
Available resources		

Table 1: List of processes and resources

Safe 就是一个process能过到完，unsafe就是不能
Format: 找Need (Max - allocated) , 找available (第一个题目给的，一直加allocated下去) / 条件是available >= need 才能加下去 // 不能就要从排

Process	Allocated A B C	Max A B C	Available A B C	Need A B C	Available >= need?
P1	0 1 2	0 3 4	2 2 2	0 1 2	ok
P2	2 0 0	3 3 4	2 3 4	1 3 4	ok
P3	3 2 1	5 6 2	4 3 4	2 4 1	no
P4	1 2 3	2 3 4	-	1 1 1	-
P5	0 4 2	1 6 6	-	1 2 4	-

Ans: The system is in an unsafe state if the sequence is P1, P2, P3, P4 and P5.

#如果是unsafe的情况，就要排多一次队伍让他safe(process的Available >= need? 都要是yes)

Process	Allocated A B C	Max A B C	Available A B C	Need A B C	Available >= need?
P1	0 1 2	0 3 4	2 2 2	0 1 2	ok
P2	2 0 0	3 3 4	2 3 4	1 3 4	ok
P4	1 2 3	2 3 4	4 3 4	1 1 1	ok
P5	0 4 2	1 6 6	5 5 7	1 2 4	ok
P3	3 2 1	5 6 2	5 9 9	2 4 1	ok
			8 11 10		

Ans: The system is in a safe state if the sequence is P1, P2, P4, P5 and P3.

Extra

Q4. Dijkstra's algorithm is a deadlock avoidance algorithm and can be used to prevent the occurrence of deadlock. Suppose at time, t_j, a snapshot of the system is taken and described as shown as Table 1.

Processes	Allocated Resources			Maximum Requirements		
	X	Y	Z	X	Y	Z
P_a	1	1	0	5	6	4
P_b	2	2	3	2	4	5
P_c	4	2	0	7	3	1

Table 1: System snapshot at time, t_j

(i) Calculate the content of the matrix *Need*.

(ii) Assuming that the available resources for X, Y and Z are 3, 1, and 2 respectively, what is the total number of resource instances for resource X, Y and Z?

(iii) Is the system in a safe state? If yes state the process sequence, if no identify the deadlock processes. Justify your answer by showing the progress of resources availability.

…